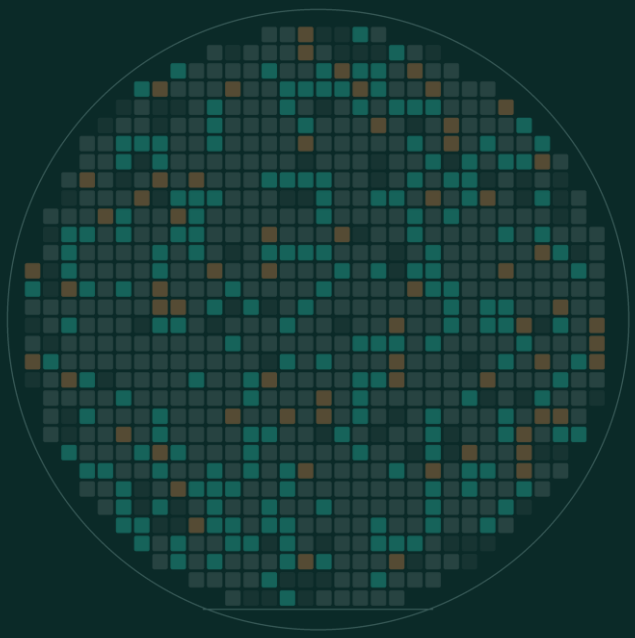
AI · CHIP TESTING

# Smart Trim & Fuse

## A prototype on synthetic data



**Project report, told simply**

Darshan Masur · Verilumen Labs shortlist project · 1 November 2026

Microcontroller · Power chip · Sensor · SoC processor · synthetic data

# Contents

1. The project in one page ... 3
The story in three parts ... 3
2. Tuning and fuses, in plain words ... 4
Wafers and chips ... 4
Why every chip needs tuning ... 4
Why fuses make it risky ... 4
How it is done today ... 4
3. The data, in plain words ... 5
A fair test: wafers the AI never saw ... 5
4. How it works: predict, check, protect ... 6
Where AI is used, and where it isn't ... 6
5. Chip by chip ... 7
5.1 Microcontroller (MCU): right first time ... 7
5.2 Microcontroller C: the AI found a hidden problem ... 8
5.3 Power chip (PMIC): one reading, four outputs ... 9
5.4 Sensor: one oven pass instead of three ... 10
5.5 SoC processor: sell what customers order ... 11
5.6 Fuse checks: rules before burning ... 12
6. What I tried, and why simple won ... 13
7. Checking that the results hold ... 14
Wafer by wafer ... 14
The SoC safety margin, chosen properly ... 14
Speed-test only the chips the AI is unsure about ... 15
Unusual chips, monitoring and running on a tester ... 15
8. How the models are saved and used ... 16
9. The website and the dashboard ... 17
The Smart Trim & Fuse website ... 17
The Power BI dashboard ... 17
10. Honest limits and next steps ... 18
11. Glossary ... 19

START HERE

# 1. The project in one page

> IN ONE SENTENCE
>
> Every chip needs a little tuning before it ships. Today the tester finds the right setting by trial and error. **Smart Trim & Fuse** lets an AI model guess the setting straight away, checks it once, and uses fixed rules to check every fuse word before it would be burned. A prototype, tested on synthetic data only.

## The story in three parts

**The problem.** No two chips come out of the factory exactly the same. So each chip is tuned on the tester (this is called **trimming**), and the chosen setting is locked into **fuses** that can be burned only once. Tuning one microcontroller takes about 20 tries today, and a wrong fuse can never be undone.

**My idea.** The tester already measures a few things about each chip. An AI model uses them to **predict** the right setting. The tester **checks** it once; if it is off, it simply falls back to today's search. Before anything is burned, fixed rules **protect** the fuses. In the simulation, a wrong guess costs a little time, not quality.

**What happened.** I used the same idea on four kinds of chips. In a simulation on synthetic wafers the AI had never seen:

| 3.9× | 3.9× | 1 pass | +35% |
|---|---|---|---|
| faster tuning<br>Microcontroller | fewer tuning tries<br>Power chip | in the oven, not 3<br>Sensor | simulated revenue<br>SoC processor |

On top of that, the AI found a hidden problem on one microcontroller version (packaging moves the voltage) and fixed it: the final-test pass rate went from 91% to 99.4% (99.0–99.8% across the only 2 wafers). And the fuse rules caught every injected error in the synthetic test file, 406 out of 406, with no false alarms. On the SoC, only the chips close to a product boundary still need the 450 ms speed test: 85% skip it, with 0 of 1,540 sold too fast (not proof of zero risk).

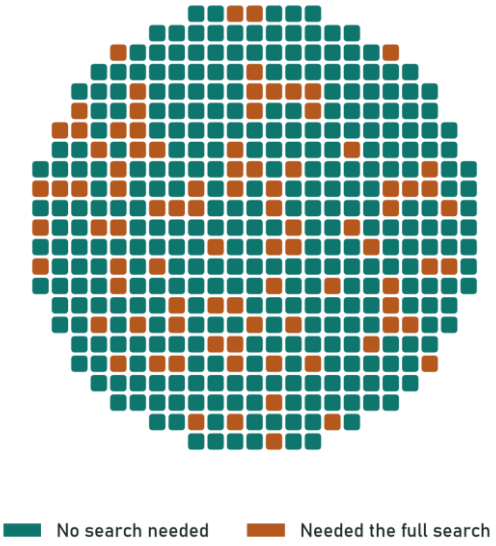
**Honest note: all data is synthetic, and every number in this report is a simulation result, not a real-world saving, yield or safety claim. This is a prototype, not a production tool, and it makes no ISO 26262 or other compliance claim. Each number's definition is in RESULTS_SOURCE_OF_TRUTH.md; what real use would need is in SAFETY_AND_LIMITATIONS.md. The next step is the same test on real wafers.**

BACKGROUND

# 2. Tuning and fuses, in plain words

## Wafers and chips

Chips are made many at a time on a round slice of silicon called a **wafer**. Each small square on the wafer is one chip (also called a **die**). In this report, every wafer picture shows each chip as a square, drawn where it sat.




*A wafer: each square is one chip. The colour shows what happened to it.*

## Why every chip needs tuning

Tiny differences in manufacturing mean one chip's voltage comes out a bit high and another's clock a bit slow. A chip has small built-in dials for this. Setting them is called **trimming**, a bit like tuning each guitar string before a concert. The setting is a small number called a **trim code**.

## Why fuses make it risky

Once the right code is found, it is stored in **fuses**: tiny switches that can be burned only once. Think of it as writing in pen, not pencil. A wrong fuse can break the chip or leave its security locks open.

## How it is done today

The tester tries a code, measures, adjusts, and measures again until the chip is on target. That works, but it is slow, it is repeated for every new chip version, and it ignores that packaging can move the value afterwards.

| Today's cost | What it means |
|---|---|
| Slow | About 20 tries to tune one microcontroller; tester time is expensive. |
| Repeated work | The same tuning is worked out again for every new chip version. |
| No second chance | Fuses are one-time, so a mistake can't be undone. |
| Drift | Packaging can shift a perfectly tuned chip off target. |

DATA

# 3. The data, in plain words

The project uses **9 datasets** for **4 kinds of chips**. For each kind there is a big main product with many wafers and one or two smaller new versions with only a few wafers, just like a real factory that keeps releasing variants.

| Dataset | Chip | What is inside | How I used it |
|---|---|---|---|
| mcu_trim_fuse | MCU | Microcontroller readings and the right trim codes | AI model |
| mcu_fuse_schema | MCU | What each fuse bit means (the fuse layout) | Reference for the rules |
| mcu_fuse_map_audit | MCU | Fuse maps to check; 1 in 4 has a planted mistake | Rules |
| pmic_trim_otp | PMIC | Power-chip readings and the right codes for its 4 outputs | AI model |
| pmic_otp_config_requests | PMIC | Customer settings for the one-time memory | Rules |
| sensor_calibration | Sensor | Sensor readings at cold, room and hot temperature | AI model |
| soc_binning_harvesting | SoC | Speed of each core, broken cores, today's product | Optimiser |
| soc_sku_demand | SoC | Price and weekly orders for Pro, Std and Lite | Optimiser input |
| memory_repair_bitmaps | SoC | Maps of broken memory cells to repair | Exact solver |

## A fair test: wafers the AI never saw

Chips on the same wafer look alike, so testing on random chips would make the AI look better than it is. Instead I hid **whole wafers** while the AI learned, then tested it only on those wafers. It is like an exam with questions you have never seen. Every model result in this report comes from this kind of test (the SoC product plan and the rule checks train nothing, so they use all the data).

> IN PLAIN WORDS
>
> All numbers come from wafers kept out of training. The data is synthetic, so these are simulated results.

THE IDEA

# 4. How it works: predict, check, protect

1. **Predict.** The tester already has a few readings for each chip: its untuned voltage, some process measurements and where it sat on the wafer. An AI model turns these into a guess of the right trim code.
2. **Check.** The tester sets that code and measures once. If the chip is on target, done. If not, it falls back to today's search, so in the simulation the worst case is today's speed, not a worse chip.
3. **Protect.** Before any fuse would be burned, fixed rules check the whole fuse word against the chip's fuse layout. A fuse can't be undone, so this step uses exact rules, not AI.

> IN PLAIN WORDS
>
> The AI only makes things faster. In the simulation, the check and the rules keep quality from depending on the AI being right.

## Where AI is used, and where it isn't

| Job | What does it | Why |
|---|---|---|
| Guess trim codes (MCU, power chip, sensor) | AI model | Learns the link between readings and the right code. |
| Check fuse maps and customer settings | Fixed rules | The right answer is written in the spec; mistakes are permanent. |
| Decide what each SoC is sold as | Optimiser | It is a planning puzzle with one best answer. |
| Repair SoC memory | Exact solver | Finds the best repair, not just a good one. |

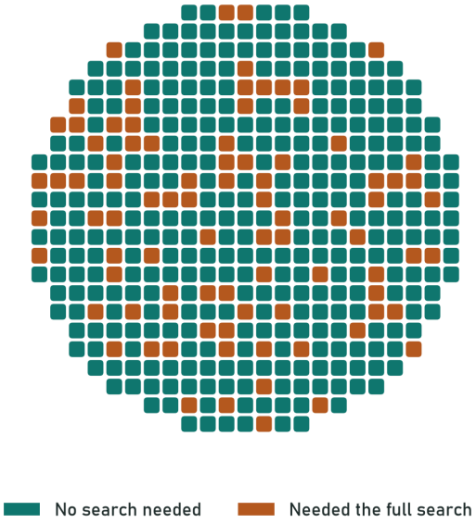
RESULTS

# 5. Chip by chip

Each part below follows the same pattern: what the chip needs, what I did, and what happened on wafers the AI never saw.

## 5.1 Microcontroller (MCU): right first time

A microcontroller has three things to tune: a voltage reference, a clock and a regulator. A **Random Forest** model predicts all three codes at once.




**83%**

of chips need no search: the AI's guess passes the check

**20 → 5**

tries to tune one chip

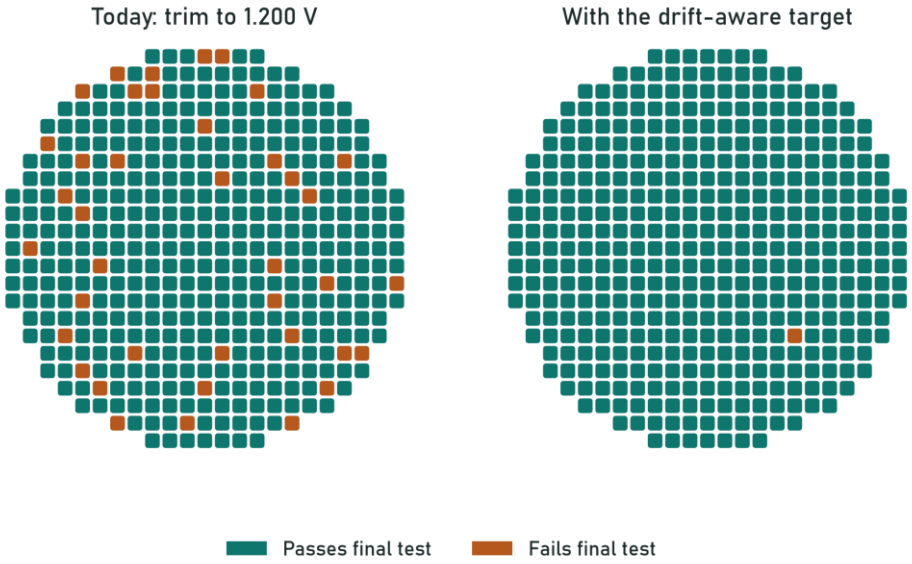
Teal chips were right first time. Copper chips were slightly off, so the tester did today's search: same quality, just slower.

IN PLAIN WORDS

Tuning time per chip drops from about 29 to 7 milliseconds, about 3.9 times faster.

## 5.2 Microcontroller C: the AI found a hidden problem

On one version, MCU-C, about 1 chip in 10 failed the final test even after perfect tuning. The cause: packaging shifts the voltage a little after tuning. So the model learns that shift and aims each chip **slightly low**, so it lands on target after packaging.




*Left: today. Right: with the drift-aware target. Copper chips fail the final test.*

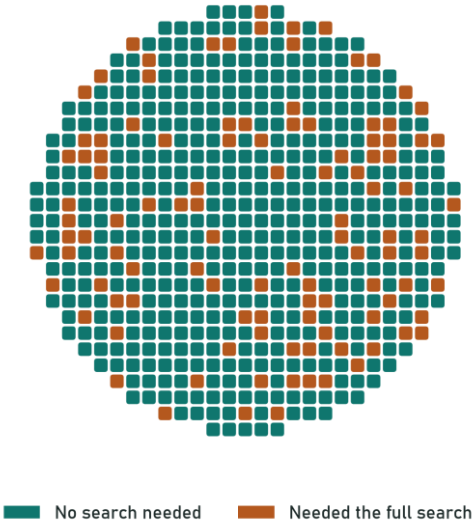
| 91% → 99.4% | 2 wafers |
|---|---|
| pass the final test | only, so an early result: checked both ways round |

This is **transfer learning**: the model learned the shift on the big MCU-A product, and 50 MCU-C chips halve the error of its prediction. New chip versions never have much data, so this is how the idea scales to every new derivative.

**Honest note:** MCU-C has only 2 wafers. Tuning on either wafer and testing on the other, with 20 different 50-chip samples, the pass rate stays between 99.0% and 99.8%. The MCU-A model alone already gets almost as far. With 2 wafers this is an early result, not a proven one.

## 5.3 Power chip (PMIC): one reading, four outputs

A power chip has four output voltages, called **rails**. I found that all four follow one reference voltage almost in a straight line, so a simple straight-line model (**Ridge**) predicts all four from a single reading.




**45 → 12**
tries to tune one chip

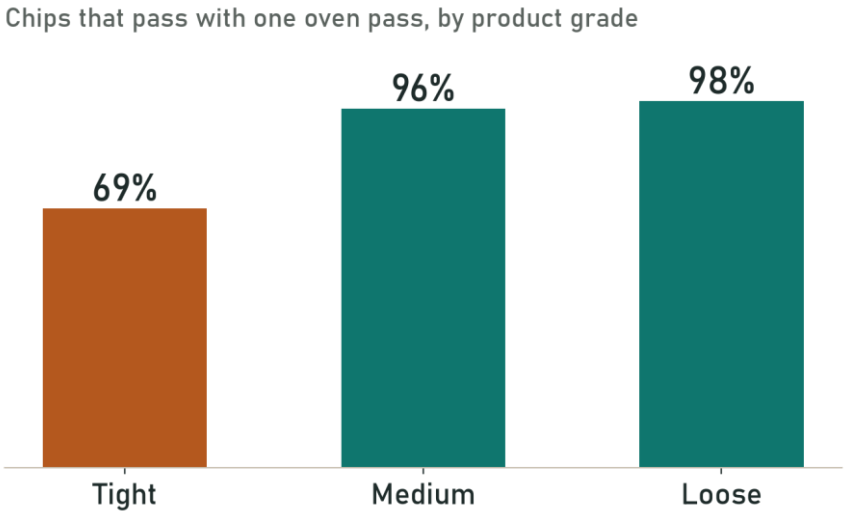
**79%**
of chips need no search

Similar on the two versions it never saw (3.5–3.9× fewer tries).

Customers also send settings for the chip's one-time memory. Fixed datasheet rules check every request: 400 synthetic requests, all judged as labelled (the labels come from the same rules, so this shows the rules are implemented correctly, not that they are complete), and every faulty part in the defect list was caught (43 of 43).

## 5.4 Sensor: one oven pass instead of three

Sensors are calibrated at three temperatures, which means three trips through an oven. I test only at room temperature and let a Ridge model predict the hot and cold behaviour.




*Share of chips that still pass with one oven pass, by product grade.*

**3 → 1**
oven passes for medium and loose grades

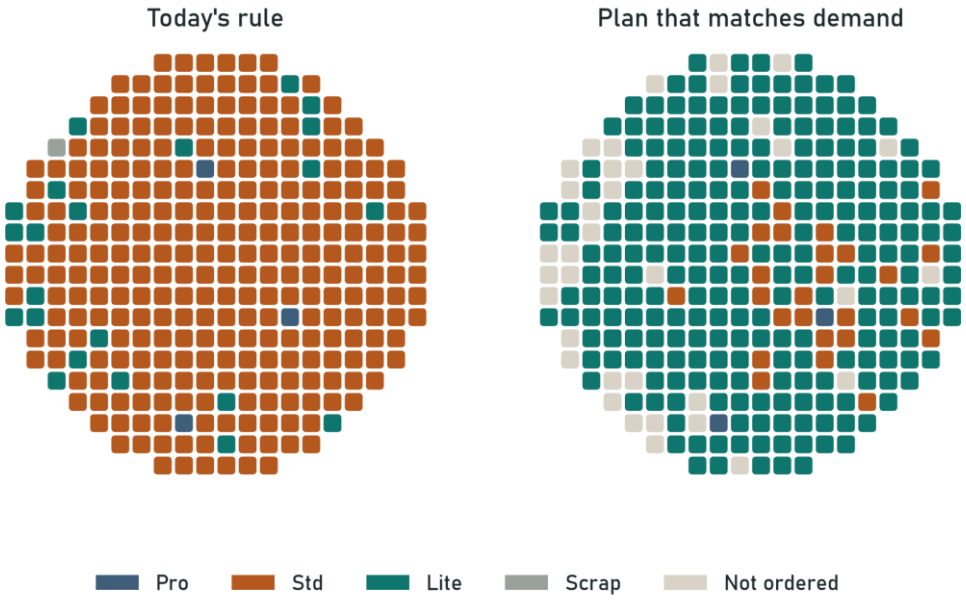
**7.2 → 2.6 s**
test time per chip

HONEST LIMIT

For the **tight** grade only about 7 in 10 chips would pass, so it keeps the full three-temperature test. Quality comes before speed.

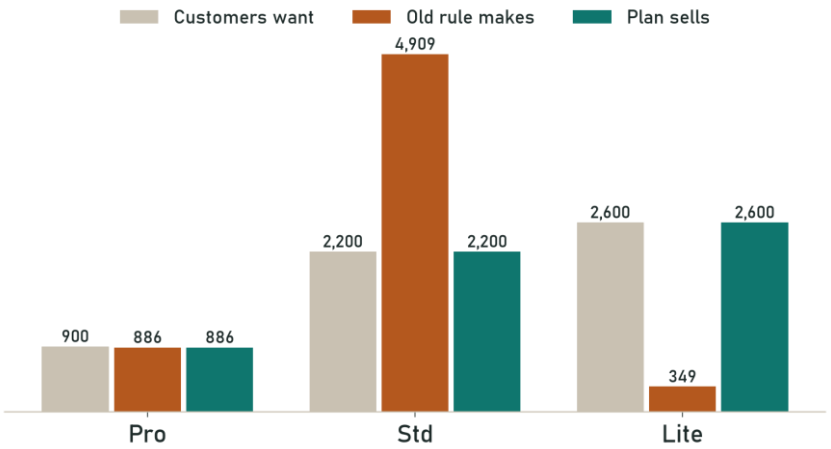
## 5.5 SoC processor: sell what customers order

The SoC is a bigger processor sold as three products: **Pro**, **Std** and **Lite**. Today's rule gives every chip the best product it qualifies for. That makes far more Std chips than customers order, while Lite runs short.




*Left: today's rule. Right: a plan that matches what customers order.*

A Std-quality chip always works as a Lite. So a planning tool (an **integer program**) decides what each chip is sold as, to match the orders. The slowest Std chips become Lite.




*Grey: what customers want. Copper: what today's rule makes. Teal: what the plan sells.*

| +35% | $357k → $480k |
|---|---|
| simulated revenue, synthetic demand | revenue a week |

Each chip also gets a 450 ms speed test before it is sold. Predicting the speed of **every** chip was too risky, but most chips are far from any product boundary. So only chips within 80 MHz of a boundary still get the speed test: **85% of chips skip it**, and none of 1,540 was sold too fast on the untouched wafers

(with 0 events the 95% upper bound is about 0.19%, so this is not proof of zero risk) (section 7 shows how the margin was chosen).

A second exact solver also repairs SoC memory better than the simple method: 17 more memory arrays saved out of 3,000.

## 5.6 Fuse checks: rules before burning

The chip's fuse layout is written in a table (the **fuse schema**). I turned it into fixed rules that check every field of a fuse word before it would be burned. A word passes only if it is exactly 64 bits, the die is found once in the sort records with the same product, and its trim fields and TS_CAL1 temperature-sensor reading match the sort records (8 rules, every field checked); otherwise it is sent for review or blocked.

| 406/406 | 0 | 100% |
|---|---|---|
| injected errors caught | good maps wrongly blocked | power-chip requests match the rule labels |

> IN PLAIN WORDS
> A fuse can't be undone, so fixed rules, not AI, make this call.

METHODS

# 6. What I tried, and why simple won

I didn't assume AI was needed; I tested it. For the microcontroller I compared a formula with six models on the same unseen wafers.I also checked newer methods that are often suggested for chip data. The table says what each is, in one line, and why it was or wasn't used.

| Method | What it is, simply | Used? |
|---|---|---|
| Formula | Today's textbook calculation | Baseline to beat |
| Ridge | A straight-line model that stays stable with little data | **Yes**: power chip and sensor; their physics is nearly a straight line |
| Random Forest | Many small decision trees that vote | **Yes**: microcontroller; it handles bends the straight line misses |
| XGBoost / LightGBM | Trees built one after another to fix errors | No: tried; no better than Random Forest here |
| Neural network (MLP) | Deep learning | No: tried; never won on data this size |
| Transfer learning | Reuse a model from a big product on a new version | **Yes**: MCU-C: the MCU-A model plus 50 chips |
| Test only when unsure | Trust the prediction only when the chip is clearly far from a limit | **Yes**: SoC: most chips skip the speed test |
| Rules | Exact checks written from the spec | **Yes**: fuse maps and customer settings |
| Integer program | An optimiser that finds the best whole-number plan | **Yes**: SoC product plan |
| Exact repair solver | Tries every repair option in a smart order | **Yes**: SoC memory repair |
| GNN / kriging | Models that use neighbours on the wafer | No: leftover errors are random from chip to chip |
| GAN | Makes fake data | No: enough real data; fake chips add nothing |
| RNN / LSTM | Models for sequences over time | No: each chip is one snapshot, not a sequence |
| Reinforcement learning | Learns by trial and reward | No: a one-shot decision with a known best answer |
| Quantile / confidence | Predicts a range, not one value | No: tested; did not save more tries |

> IN PLAIN WORDS
>
> Pick the simplest method that clearly works. Where there is an exact right answer, use rules or an optimiser.

CHECKING

# 7. Checking that the results hold

Notebook 06 asks the questions a test engineer would ask before trusting the numbers above. It changes no model; it shows how far each result can be trusted.

## Wafer by wafer

Each wafer of the main product was held out once: the model was retrained on all the other wafers and scored on that one. So each result comes with a **range**, from the hardest wafer to the easiest, not one lucky number.

| Result | One split | Middle wafer | Hardest to easiest wafer | Wafers |
|---|---|---|---|---|
| MCU: chips needing no search | 83.0% | 83.4% | 67.0% to 90.7% | 15 |
| MCU: tuning speed-up | 3.9x | 3.9x | 2.9x to 4.7x | 15 |
| Power chip: fewer tries | 3.9x | 3.9x | 3.6x to 4.0x | 15 |
| Power chip P2/C7 (new versions): fewer tries | 3.9x | 3.9x | 3.5x to 3.9x | 5 (never trained on these versions) |
| Sensor: pass with 1 oven pass (Medium) | 96.0% | 96.2% | 95.0% to 97.3% | 15 |
| Sensor: pass with 1 oven pass (Tight) | 69.3% | 68.8% | 66.4% to 70.8% | 15 (tight grade keeps the 3-temperature test) |
| Sensor 8G/16G (new versions), Medium | 96.0% | 95.9% | 95.7% to 96.6% | 5 (2-3 wafers per version) |
| SoC: revenue from the plan, per wafer | 34.7% | 41.6% | 33.1% to 58.4% | 20 (weekly plan over all wafers: +34.7%) |
| MCU-C: final-test pass with drift fix | 99.8% | 99.4% | 99.0% to 99.8% | 2 (only 2 wafers: early result) |

Most results barely move. The microcontroller moves the most: two wafers are clearly harder than the rest, so the 83% is the middle of a 67-91% range.

## The SoC safety margin, chosen properly

The speed margin must be chosen on wafers that are **not** the test wafers, or the test result isn't independent. So the training wafers were split again: models were fitted on 11 wafers, the margin chosen on 4 **validation** wafers, and only then were 5 untouched wafers scored.

- **"At most 0.1% escapes on validation"** picked 60 MHz. On the untouched wafers it gave **0.32%** escapes: worse than validation.

- **"Zero escapes on validation"** picked **80 MHz**: 0 of 1,540 escapes on the untouched wafers. So the stricter rule is the one to use.

## Speed-test only the chips the AI is unsure about

Each chip's product is worked out with its predicted speed minus and plus 80 MHz. If both give the same product, the chip is far from any boundary and is sold on the prediction. If not, it gets the normal speed test.

| 85% | 0 | 100% |
|---|---|---|
| of SoC chips skip the 450 ms speed test | sold too fast on 1,540 untouched chips | of the revenue of full testing |

## Unusual chips, monitoring and running on a tester

- **Out-of-range check:** a chip with a reading outside what the model learned from goes straight to today's method. It flags almost no chips of the main products, but 3 in 4 MCU-C chips (a different oxide thickness), so ranges are kept per product version and learned during shadow mode.
- **The fallback rate is the alarm:** if more chips fail their check, something has changed. A simulated tester fault (a voltmeter reading 4 mV high) shows up clearly; in the simulation, quality is protected because the check catches every wrong code.
- **Predict a whole wafer at once:** about 0.2 ms per chip, so it runs ahead of the test instead of slowing it.
- **Tuned for tester time, not just accuracy:** both ways pick practically the same model, so nothing changed.

> HONEST LIMIT
> These checks show the methods are stable on the synthetic data. They can't prove the same on real wafers: that needs the same checks on real data, in shadow mode.

BUILD

# 8. How the models are saved and used

Each chip family has its own Colab notebook. The notebook cleans the data, compares the models, picks the winner, and saves it to one file with **joblib** (a Python library for saving models).

1. **Notebook 00** looks at all 9 datasets and cleans them.
2. **Notebooks 01 to 04** train one chip family each (MCU, power chip, sensor, SoC) and save a model file.
3. **Notebook 05** checks the advanced methods (graph models, GAN, reinforcement learning and others).
4. **Notebook 06** checks the results wafer by wafer, chooses the SoC margin on validation wafers, and saves **evaluation_06.json** for the website.
5. The **website** loads the same four model files, so it shows exactly what the notebooks measured.

| Model file | Chip | What is inside |
|---|---|---|
| trim_models.joblib | Microcontroller | Random Forest for the 3 trims + the drift model |
| pmic_models.joblib | Power chip | Ridge models for the 4 rails + the limits for checking customer settings |
| sensor_models.joblib | Sensor | Models for hot and cold behaviour + the spec limits |
| soc_models.joblib | SoC | Speed models (kept for comparison), the safety margin and the product table |

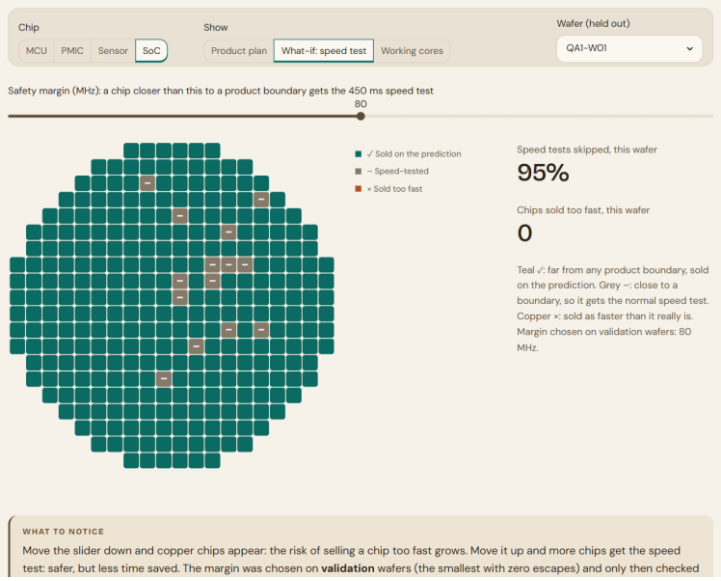
SHOW IT

# 9. The website and the dashboard

## The Smart Trim & Fuse website

A small website (built with Streamlit) runs on a normal laptop. It has five short pages:

| Page | What it shows |
|---|---|
| Overview | The idea in three steps and the four main results |
| Wafer lab | Wafer maps for every chip type, plus what-if sliders: the SoC safety margin and the checks before falling back |
| Live demo | One chip in 4 steps (measure, predict, check, save), with Typical and Tricky chip buttons |
| Results | Before → after for each chip with its range across wafers, baselines, model cards |
| Batch check | Upload a test file; every chip gets a result |




*The Wafer lab what-if: teal ✓ chips are sold on the prediction, grey – ones get the speed test.*

## The Power BI dashboard

The dashboard tells the same story in four pages, so a manager can read it without the website:

| Page | What it shows |
|---|---|
| 1 · Overview | The problem, the idea, the four main results and a before → after table |
| 2 · Wafer maps | MCU, MCU-C before and after the drift fix, and the power chip |
| 3 · SoC plan | Today's rule vs the plan on the same wafer, the revenue, what customers ordered, and the speed tests skipped |
| 4 · Sensor, safety & methods | Sensor grades, the fuse-safety checks, the methods tried and the 9 datasets |

NEXT

# 10. Honest limits and next steps

- **The data is simulated.** Next: run the same held-out-wafer test on real factory data.
- **Shadow mode first.** The AI would run beside today's flow for a while, so every decision can be compared before it is trusted.
- **Always check before burning.** The one-time check and the fuse rules stay, so in the simulation a wrong guess costs time, not quality.
- **Prototype, not a production tool. No ISO 26262 or other compliance is claimed, and nothing here may program real fuses. Real use would need real-data validation, shadow mode, agreed escape targets, read-back verification and change control (see SAFETY_AND_LIMITATIONS.md).**
- **New chip versions.** Fine-tune with a few chips, like MCU-C, learn their input ranges in shadow mode, then check on unseen wafers before switching on. MCU-C itself has only 2 wafers: an early result.
- **Tight sensor grade.** Keeps the full three-temperature test.
- **SoC speed test.** Still runs for the chips close to a product boundary (about 1 in 7); predicting every chip's speed would need too big a safety margin.

WORDS

# 11. Glossary

| Word | Meaning |
|---|---|
| Wafer | A round slice of silicon with hundreds of chips on it |
| Die | One chip on the wafer |
| Trim / trimming | Tuning a chip's voltage or clock with a small setting |
| Trim code | The small number that sets the tuning |
| Fuse / OTP | A one-time switch that stores settings; it can't be undone |
| Fuse map / fuse schema | All the fuse bits of a chip / the table that says what each bit means |
| Derivative | A new version of a chip, built from the same design |
| MCU | Microcontroller: a small computer on one chip |
| PMIC | Power-management chip: makes the voltages other chips need |
| Rail | One output voltage of a power chip |
| SoC | System-on-chip: a bigger processor |
| Pro / Std / Lite | The three SoC products, from fastest to simplest |
| Drift | A shift in a chip's value after packaging |
| Held-out wafer | A wafer hidden from the AI while it learns, used only for testing |
| Leave-one-wafer-out | Hold out each wafer once and retrain on the rest, to see the range across wafers |
| Validation wafers | Wafers used to choose a setting (like the safety margin), kept apart from the test wafers |
| Shadow mode | The AI predicts beside today's flow, which still decides, so every decision can be compared |
| Transfer learning | Reusing a model from a big product on a new, smaller one |
| Ridge / Random Forest | The two simple AI models that won |
| Integer program | An optimiser that finds the best plan with whole numbers |
| joblib | The Python library used to save and load the models |